ポートフォリオ

実務経験

2019年4月～2023年1月　｜　フロントエンドエンジニア

中国のIT企業2社にて、約4年間、正社員としてWebシステム開発に従事しました。Vue.jsを中心にReactも 使用し、業務用管理画面、大型ディスプレイ向けダッシュボード、企業サイトを開発。プロトタイプ・デザインに基づく画面実装からAPI連携、テスト、保守・改修まで担当しました。

フロントエンドエンジニアとして活かせる経験

仕様・デザインをもとに業務画面を構築し、APIから取得したデータを表・グラフ・地図・映像として表示する実装経験があります。新規画面の開発から既存機能の改修まで担当し、複数案件を並行して進める環境で品質と納期を意識して取り組みました。

主な使用技術：Vue.js、React、JavaScript、HTML、CSS、Ant Design、Element UI、ECharts、GitLab

業務システムとダッシュボードの開発

北京微分格科技有限公司　2020年4月～2023年1月

銀行、エネルギー企業、自動車会社向けのデジタル金融プラットフォーム、倉庫管理・監視システム、IoTプラットフォームなどのフロントエンド開発に携わりました。複数案件でフロントエンド開発の主要部分を担当しました。

管理画面では、Ant Design・Element UIを活用し、一覧、検索、入力フォームを実装。バックエンドAPIと連携したデータ取得・更新処理に加え、既存画面のレイアウト調整、不具合修正、仕様変更にも対応しました。

代表的な実装例としての倉庫監視ダッシュボード

倉庫の稼働状況を確認するため、グラフ、監視映像、リアルタイム情報を一つの画面に集約したダッシュボードを実装しました。業務用管理画面に加え、大型ディスプレイで状況を確認する画面の開発も経験しています。

関連する案件では、EChartsによるグラフ表示、OpenLayersによる地図表示、WebSocketによる監視情報のリアルタイム更新、hls.js・flv.jsによる映像再生を担当しました。データ表示と映像表示の両方を扱うフロントエンド開発を経験しています。

企業サイトと社内管理システムの開発

江西全栈科技有限公司　2019年4月～2020年3月

Vue.js、JavaScript、CSSを用いて、企業ホームページや社内向け管理システムを開発しました。要件確認、画面設計、実装、テスト、納品後の保守に携わり、スマートフォン向けのレスポンシブ対応やページパフォーマンスの最適化、クライアントの仕様変更に対応しました。

個人開発

ことね-言語シャドーイング

開発期間 2025年12月～現在 ｜ 2026年11月時点

プロジェクト概要

自身の日本語学習で感じた課題をもとに、動画・字幕を活用したシャドーイング練習アプリを開発しています。SwiftによるiOSアプリに加え、React／TypeScriptによるWeb管理画面と、Cloudflare WorkersによるバックエンドAPIを実装しています。

開発の背景

動画で学習する際に、分からない表現を調べたり、前回の再生位置に戻ったりする操作が練習の中断につながっていました。動画再生、字幕確認、単語検索、AI解説、学習記録をアプリ内にまとめ、練習を続けやすい操作の流れを目指しています。

担当範囲

機能の検討、画面設計、iOS・Webの画面実装、API開発、データベースの設計・実装を担当。Web管理画面では、動画や練習記録などの学習データを一覧から検索・編集できる機能を実装しています。

技術構成

領域	使用技術と用途
Webフロントエンド	React、TypeScript、Vite、React Router Tailwind CSS、shadcn/ui、TanStack Table
iOSアプリ	Swift、SwiftUI、AVKit／AVFoundation、Swift Charts
API・データ管理	TypeScript、Cloudflare Workers、Cloudflare D1（SQLite）
認証・決済	Sign in with Apple、StoreKit、Stripe／Apple Pay

Web管理画面の実装

学習データを扱う管理画面として、動画・文法・プレイリストなどの一覧画面と編集機能を実装しています。以下は、主に動画管理画面での実装例です。

一覧から対象データを探して編集する

TanStack Tableを使用し、検索とページネーションを実装しました。検索条件を変更した際は先頭ページへ戻し、絞り込み後の結果を確認できるようにしています。文法管理画面にはJLPTレベルによる絞り込みも設けています。

項目数の多い表は横スクロールに対応し、編集・削除の操作列を右端に固定しました。画面幅に応じて検索欄などの配置を切り替え、編集はダイアログ内で行える構成です。

入力と保存結果を分かりやすく伝える

動画の編集フォームでは、タイトルの未入力や数値項目の不正な入力をチェックします。保存中はボタンを無効化し、完了・失敗を通知します。APIの更新成功後は対象行のstateを更新し、ページ全体を再読み込みせずに変更内容を反映します。

削除操作には確認ダイアログを設け、APIの削除成功後に対象データを一覧から取り除きます。処理中の状態も管理し、操作の重複を防ぐようにしています。

通信状況に応じて表示を切り替える

動画一覧では、読み込み中、取得失敗、データなし、取得成功を分けて表示します。取得失敗時には再読み込みボタンを用意し、利用者が画面から復帰できるようにしました。動画と字幕のアップロードでは、ファイル形式の確認、処理中の表示、完了・エラーの通知を実装しています。

画面とAPI通信の責務を分ける

ページごとの画面実装とAPI呼び出しを別ファイルに分け、動画データや更新項目をTypeScriptの型で定義しています。共通のAPI通信処理で認証ヘッダーの付与と401応答時の処理を扱い、画面ごとの実装が重複しないようにしています。

ログイン後に元の操作へ戻る

React Routerで画面を構成し、認証確認中・未ログイン・通信失敗を区別しています。未ログイン時はログイン画面へ遷移し、認証後に元のページへ戻れるように遷移先を保持しています。



Web管理画面 1

クレジットチャージ

チャージするクレジットを選び、Apple Pay でお支払いください。

現在の残高
31,094 クレジット

テスト決済モードです。実際の請求は発生しません。

商品を選択

1,000 クレジット 日本語学習クレジット	¥50
2,000 クレジット 日本語学習クレジット	¥100
4,000 クレジット 日本語学習クレジット	¥200
10,000 クレジット 日本語学習クレジット	¥500

お支払い金額 **¥50**

Pay

お支払いとクレジットの追加は Stripe の決済確認後に処理されます。

Web管理画面 2



Web管理画面 3

iOSアプリとAPIの連動

動画と字幕を使った学習画面

動画と字幕を連動させた再生画面を実装し、再生速度の調整、再生位置の保存・復元、字幕の表示設定に対応しています。字幕中の単語から辞書を開く機能や、単語・字幕を対象としたAI解説を組み込み、学習中に必要な情報を確認できるようにしています。

AIの回答待ちと通信エラーへの対応

AI解説はストリーミングで受信し、生成された部分から順に表示します。生成中の表示、追加質問の入力制御、エラー表示を設け、通信に失敗した場合は同じリクエストを再確認できるようにしています。

API側ではリクエストIDと処理状態を管理し、同一リクエストの再確認時には保存済みの結果を返す仕組みを実装しました。画面の再試行操作とサーバー側の処理を結び付け、クレジットの重複消費を防ぐ設計にしています。



iOSの学習画面

学習状況の可視化

学習時間を記録し、日・週・月などの期間を切り替えて確認できるグラフをSwift Chartsで実装しています。再生機能だけでなく、学習の継続状況を振り返る画面も用意しています。

認証と購入情報の管理

Appleでのサインインとログイン状態の管理、StoreKitによるアプリ内購入、Stripe経由のApple Pay決済を実装しています。Cloudflare D1でユーザー情報、クレジット残高・購入履歴、AIへの質問・回答履歴を管理し、アプリとWebからAPIを通じて利用する構成です。

テストで確認する処理

認証、AIリクエスト、クレジット処理、決済などに対するテストコードを用意しています。AIリクエストでは、同じIDでの再送信による重複消費の防止や、異常な入力を受けた際に残高が変わらないことなどを確認対象としています。

この開発で深めたこと

画面の表示だけでなく、入力、通信、保存、エラーからの復帰までを一連の操作として考える経験を積みました。APIとデータ管理も実装することで、フロントエンドが必要とするデータや処理状態を整理し、それに合わせて画面の動作を設計しています。